Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3069 Computación Paralela y Distribuida - Sección 20, Semestre II 2026



Laboratorio 12

Speedup y eficiencia

Integrantes:

Nina Nájera Marakovits #231088

Luis Palacios #23933

1. Equipo y mediciones

Se estudiaron cuatro experimentos con C y OpenMP. Las partes 1 y 4 se midieron en el equipo de Nina y las partes 2 y 3 en el de Luis. Cada tiempo corresponde a la mediana de cinco repeticiones. Se utilizó la versión secuencial sin OpenMP del mismo trabajo como referencia.

Dato	Nina	Luis
CPU	Intel Core i7-1165G7	Intel Core i5-12450H
Núcleos físicos	4	8
Procesadores lógicos	8	12
Rendimiento y eficiencia	Sin división P y E	4 P y 4 E
Hyper-Threading	Sí, dos hilos por núcleo	Sí en los 4 P, no en los E
Entorno	Windows 11, GCC 10.3.0	WSL2 Ubuntu 24.04, GCC 13.3
Máximo de hilos usado	8	12

Se usó optimización O2. Ambos equipos estaban conectados a corriente. En el equipo de Nina no se controlaron afinidad, frecuencia, temperatura ni procesos de fondo. En el de Luis había aplicaciones abiertas. Se conservaron corridas adicionales para revisar su variación. Por eso los tiempos de ambos equipos se analizan por separado. La resolución del reloj de Nina fue aproximadamente de un milisegundo. En WSL se mostraron 12 procesadores virtuales y una organización de 6 núcleos por 2 hilos que no representa la división física P y E.

Los datos originales de Luis corresponden al programa inicial. Los de Nina corresponden a ciclos con nowait y barreras explícitas. Se compararon ambas versiones y la diferencia fue visible principalmente en sumas pequeñas. La versión unificada conserva las barreras explícitas y las nuevas corridas se guardan aparte. Los resultados originales no se reemplazaron ni se presentan como mediciones de la versión nueva.

2. Parte 1. Problema fijo

2.1. Predicción y resultados

Antes de medir se esperaba un speedup cercano a 4 con 8 hilos y $f_p = 1$. Para $f_p = 0{,}90$ se obtuvo una predicción ideal de 4.706 y una expectativa práctica cercana a 3.077. Se esperaba que la curva con $f_p = 0{,}75$ se aplanara. Se ejecutaron 400000 unidades de trabajo con el mismo tamaño para todos los hilos.

f_p	S_2	S_4	S_8	E_8	p con F máxima
1.00	1.928	3.487	6.798	85.0 %	8
0.95	1.908	3.147	4.914	61.4 %	7
0.90	1.817	2.729	3.150	39.4 %	6
0.75	1.635	2.197	2.725	34.1 %	2
0.50	1.342	1.566	1.782	22.3 %	1

2.2. Respuestas

1. Con $f_p = 1$ se obtuvo $S = 6{,}798$, menor que 8 y mayor que la predicción cercana a 4. Hyper-Threading comparte recursos de cada núcleo y la frecuencia puede cambiar al usar más hilos. Crear y sincronizar equipos también agrega costo y corresponde a η_c. No se midieron frecuencias ni afinidad para atribuir un porcentaje a cada causa.

2. Con $f_p = 0{,}75$ el speedup fue 1.635 con 2 hilos y 2.725 con 8. El segundo hilo agregó aproximadamente 0.595 al speedup medido con un hilo. El octavo agregó 0.031 frente a 7. La ganancia disminuye por la parte que sigue siendo secuencial.

3. Si T_1 es el tiempo en un núcleo, la parte secuencial tarda $T_1[1 - f_p]$ y la paralela ideal tarda $T_1 f_p/p$. Por lo tanto:

$$T_p = T_1 \left[1 - f_p + \frac{f_p}{p}\right], \qquad S_p = \frac{1}{1 - f_p + f_p/p}.$$

   El factor T_1 se cancela al dividir los tiempos.

4. Para $f_p = 0{,}90$ y 8 hilos, el modelo predice 4.706 y se midió 3.150, un 33.1 por ciento menos. La expectativa práctica de 3.077 quedó cerca, pero ese acuerdo no demuestra que existan cuatro núcleos efectivos constantes.

5. Cuando p crece sin límite, S tiende a $1/[1 - f_p]$. Con $f_p = 0{,}90$ el límite es 10 y con 1000 hilos la predicción es 9.911.

6. Al despejar se obtiene:

$$f_p = \frac{1 - 1/S_p}{1 - 1/p}.$$

   Con el speedup medido de 3.150 y 8 hilos se obtiene 0.780. Es una fracción efectiva que reúne costos ignorados por el modelo. La diferencia con 0.90 puede incluir η_c y cambios en la velocidad efectiva del hardware; no prueba una nueva parte secuencial del código.

7. Para $f_p = 0{,}90$, la mayor efectividad y el menor tiempo ocurrieron con 6 hilos. Se obtuvo $T_p = 0{,}192$ s, $S = 3{,}510$, $C = 1{,}152$ hilo-segundos y $F = 3{,}047$. El ejemplo de 16 números con óptimo en 4 hilos usa otro trabajo y otros costos. Con $f_p = 0{,}75$, la efectividad máxima ocurrió con 2 hilos y el menor tiempo con 8.

8. Para un programa de 20 segundos con 2 segundos secuenciales, 64 núcleos darían idealmente $T_p = 2 + 18/64 = 2{,}28125$ s. Se obtiene $S = 8{,}767$, $E = 13{,}70\,\%$, $C = 146$ hilo-segundos y $F = 0{,}06005$. Se reduce el tiempo, pero no 64 veces. El máximo ideal de efectividad ocurre con 9 núcleos: $T_p = 4$ s, $S = 5$, $C = 36$ y $F = 0{,}13889$. Comprar más núcleos solo por la rapidez requiere considerar ese costo y la lectura secuencial.

3. Parte 2. Referencia y granularidad

3.1. Predicciones

Se esperaba que OpenMP con un hilo fuera entre 0 y 3 por ciento más lento en Amdahl. Para la suma se predijo que la paralela empezaría a ganar entre 3000 y 8000 números. Con 16 números y 12 hilos se esperaba una pérdida de entre 100 y 300 veces. Para los tamaños grandes se esperaban speedups de 4 a 8, limitados por hardware y memoria.

3.2. Respuestas 2a

1. En Luis, Amdahl con un hilo midió $T_s = 1{,}521067$ s y $T_p = 1{,}478568$ s. La paralela fue 2.79 por ciento más rápida en esa corrida. En cinco corridas adicionales, la mediana de la diferencia relativa fue 1.23 por ciento más lenta. El signo cambió entre corridas, por lo que no se demuestra una ventaja estable. En Nina se midieron 0.673 y 0.671 segundos: una diferencia de 2 ms y 0.30 por ciento. Los 2 ms superan la resolución aproximada de 1 ms, pero no descartan variación del sistema.

2. Usar el paralelo con un hilo como referencia infla S cuando ese tiempo es mayor que T_s y lo desinfla cuando es menor. En suma con 1024 números y 2 hilos, $1015/1443 = 0{,}703$ con referencia honesta, pero $1532/1443 = 1{,}062$ con la referencia paralela de un hilo. Así se puede presentar una pérdida como ganancia.

3.3. Respuestas 2b

3. Con 16 números y 12 hilos, $T_s = 5$ ns y $T_p = 21736$ ns. Se obtiene $S = 0{,}000230$ y la paralela resulta aproximadamente 4347 veces más lenta. La predicción quedó muy por debajo del costo observado. El valor usa tiempos impresos redondeados a nanosegundos.

4. El modelo de dos fases supone un costo pequeño o despreciable al repartir, combinar y sincronizar. En la máquina, 16 números con 4 hilos tardaron 1708 ns. Ese costo domina unas pocas sumas y corresponde a η_c.

5. OpenMP debe activar el equipo, repartir iteraciones, inicializar resultados privados, combinar la reducción y esperar a los hilos. Con 16 números el cálculo útil es mínimo y dominan esos costos. La versión con barrera explícita también tiene el cierre obligatorio

de la región; por eso no se interpreta la diferencia con el programa inicial como el costo aislado de una sola barrera.

6. Las tres corridas con tamaños intermedios ubicaron la primera ventaja cerca de 4096 números para 2, 4 y 8 hilos. Con 12 hilos apareció cerca de 16384. La predicción de 3000 a 8000 fue razonable para menos hilos y optimista para 12. Se debe paralelizar cuando el trabajo útil por región supera suficientemente el costo de reparto y sincronización.

7. Con 16777216 números, el mejor speedup oficial fue 4.52 con 8 hilos y cayó a 3.21 con 12. El arreglo ocupa unos 134 MB y los hilos comparten el acceso a RAM. Se estimó una lectura conjunta de unos 29 GB/s, pero no se midió el ancho de banda máximo por separado. La competencia por memoria es una explicación compatible con los datos, no una demostración del único límite. No queda descrita por completo en las cuatro fugas.

4. Parte 3. Desbalance

4.1. Predicción y resultados

Se predijo que el último hilo con static y 4 hilos recibiría aproximadamente $7/16$ del trabajo y que el speedup sería cercano a 2.3. Con 12 hilos se esperaba static entre 4.5 y 5.5, dynamic entre 7 y 9 y guided entre 7 y 8.5. Se esperaba que dynamic ganara o empatara con guided.

Hilos	S static	S dynamic	S guided
2	1.46	1.80	2.01
4	2.51	3.93	4.06
8	4.58	8.00	5.79
12	3.74	7.49	4.46

4.2. Respuestas

1. En la corrida oficial gana dynamic con $S_{12} = 7{,}49$, frente a 3.74 de static y 4.46 de guided. Es cerca de dos veces el speedup de static. Al considerar la oficial y cinco adicionales, las medianas fueron 3.95, 8.03 y 8.04. Dynamic y guided quedaron prácticamente empatados. Static quedó por debajo de su predicción y las otras políticas cerca de lo esperado.

2. Al aproximar el costo por una recta, el último bloque ocupa una fracción $[2p - 1]/p^2$ del trabajo. Con 2 hilos corresponde a $3/4$ y predice $S = 1{,}333$. Con 4 corresponde a $7/16$ y predice $S = 2{,}286$. Se midieron 1.46 y 2.51 frente a la referencia secuencial pura. Los 40000 costos discretos dan fracciones cercanas a 0.7499 y 0.4374. El modelo supone hilos de igual velocidad y no incluye overhead.

3. Los primeros hilos terminan sus bloques baratos y esperan al último. Se pagan esos hilo-segundos aunque no produzcan trabajo útil. Con 4 hilos, static tuvo eficiencia 0.628 y dynamic 0.982. La espera explica la fuga de desbalance η_b.

4. Dynamic pide bloques durante la ejecución y eso agrega acceso a un contador compartido y costo de reparto. Es útil cuando las iteraciones tienen costos diferentes y cada bloque tiene trabajo suficiente. En sumas pequeñas ese costo puede superar el ahorro. Static evita pedir cada bloque en ejecución, aunque tampoco tiene costo total cero.

5. Con núcleos P y E, igual número de iteraciones no significa igual tiempo. Los núcleos tienen distinta capacidad y los hilos de Hyper-Threading comparten recursos. Un hilo lento puede retrasar todo el equipo aun con iteraciones de igual costo. Sin registrar afinidad no se puede afirmar en qué núcleo corrió cada hilo ni que usar hasta 8 hilos evitó los núcleos E.

5. Parte 4. Problema creciente

5.1. Predicción y resultados

Antes de medir se calculó que el modelo fijo con $f_p = 0{,}83$ tiene límite 5.882. Se esperaba que ese límite no describiera el problema creciente, que T_s aumentara y que T_p cambiara menos. Se mantuvieron 20000 unidades secuenciales y se agregaron 100000 unidades paralelas por hilo.

p	T_s s	T_p s	s	S medido	S escalado
1	0.232	0.203	0.182	1.143	1.000
2	0.367	0.207	0.179	1.773	1.821
3	0.549	0.218	0.183	2.518	2.633
4	0.710	0.227	0.150	3.128	3.551
5	0.888	0.233	0.167	3.811	4.330
6	1.046	0.246	0.159	4.252	5.207
7	1.208	0.234	0.162	5.162	6.026
8	1.382	0.262	0.126	5.275	7.118

5.2. Respuestas

1. T_s pasó de 0.232 a 1.382 segundos y T_p de 0.203 a 0.262 al pasar de 1 a 8 hilos. El trabajo total creció de 120000 a 820000 unidades. La referencia secuencial procesa todo el aumento y la paralela lo reparte. Se cumplió la tendencia prevista, aunque T_p no fue constante.

2. Con 8 hilos se midió $S = 5{,}275$. El modelo fijo con $f_p = 0{,}83$ predice 3.653 para 8 hilos. No es el mismo problema que en la parte 1: la parte paralela aumenta mientras la

secuencial se mantiene. Por eso la fracción paralelizable del problema completo también cambia.

3. Se calculó $s = T_{\text{parte sec}}/T_p$ con la fase secuencial de la misma repetición mediana. La tabla muestra que s cambió de aproximadamente 0.182 a 0.126. No se mantuvo constante y hubo variación entre filas.

4. En el modelo ideal, el trabajo secuencial tarda sT_p y el paralelo, al hacerlo en un núcleo, tarda $p[1 - s]T_p$. Entonces:

$$T_s^{\text{ideal}} = sT_p + p[1 - s]T_p, \qquad S^{\text{ideal}} = s + p[1 - s] = p - s[p - 1].$$

   Con 8 hilos y $s = 0{,}125954$ se predice 7.118 frente a 5.275 medido. El modelo supone la misma velocidad efectiva del trabajo en un hilo secuencial y en cada hilo paralelo. El hardware compartido y los costos de ejecución impiden una coincidencia exacta.

5. La parte 1 pregunta cuánto más rápido se resuelve el mismo problema, como renderizar una imagen fija. La parte 4 pregunta cuánto puede crecer el problema en un tiempo parecido, como renderizar más imágenes al disponer de más núcleos.

6. Cálculos

Para Amdahl con $f_p = 0{,}90$ y 4 hilos se midieron $T_s = 0{,}674$ s y $T_p = 0{,}247$ s:

$$S = \frac{0{,}674}{0{,}247} = 2{,}728745, \qquad E = \frac{2{,}728745}{4} = 0{,}682186,$$

$$C = 4 \cdot 0{,}247 = 0{,}988, \qquad F = \frac{2{,}728745}{0{,}988} = 2{,}761888.$$

Para dynamic con 4 hilos en desbalance se midieron $T_s = 0{,}153325$ s y $T_p = 0{,}039029$ s:

$$S = \frac{0{,}153325}{0{,}039029} = 3{,}928489, \qquad E = \frac{3{,}928489}{4} = 0{,}982122,$$

$$C = 4 \cdot 0{,}039029 = 0{,}156116, \qquad F = \frac{3{,}928489}{0{,}156116} = 25{,}163911.$$

El costo se expresa en hilo-segundos y la efectividad en su inverso.

7. Síntesis y límites

Parte	Fuga principal	Evidencia
1	η_f, parte secuencial	Con 8 hilos, la eficiencia baja de 84.97 por ciento para $f_p = 1$ a 22.27 para $f_p = 0{,}50$.
2	η_c, comunicación y sincronización	Con 16 números y 12 hilos, 21736 ns paralelos frente a 5 ns secuenciales.
3	η_b, desbalance	Con 4 hilos, eficiencia 0.628 en static frente a 0.982 en dynamic.
4	η_f y costos compatibles con η_c	Con 8 hilos, la fase secuencial ocupa 12.60 por ciento de T_p y el modelo escalado predice 7.118 frente a 5.275 medido.

No se agregó trabajo redundante deliberado y no hay una medición que permita atribuir una pérdida concreta a η_r. Tampoco se puede separar cada fuga usando solo el tiempo total. La frecuencia dinámica, Hyper-Threading, la caché, el ancho de banda de memoria y la virtualización afectan el rendimiento y no quedan descritos por completo en las cuatro fugas.

La variación no se limitó a 12 hilos. Por ejemplo, guided con 8 hilos tuvo una corrida con speedup 5.79 frente a una mediana cercana a 7.44, una diferencia de aproximadamente 22 por ciento. Las conclusiones usan las repeticiones y evitan atribuir toda la diferencia a una sola causa.

7.1. Reto superlineal

Se usó la misma suma AVX2 en la referencia y en los bloques paralelos. Para 262144 números, las cinco corridas dieron medianas de speedup 2.57 con 2 hilos y 5.80 con 4. Todas superaron el número de hilos. Repartir un arreglo de 2 MiB en bloques que caben en cachés más pequeñas es una explicación compatible con ese resultado. No se registraron afinidad ni contadores de caché, por lo que la causa no queda demostrada de forma exclusiva. El programa original no garantiza impedir un speedup superlineal; los casos pequeños por encima del ideal deben revisarse con repeticiones.

Se concluye que agregar hilos no garantiza una mejora proporcional. Se necesita suficiente trabajo por región, una referencia honesta y un reparto adecuado. Las mediciones permiten observar los límites del problema fijo y la diferencia al aumentar el trabajo con los hilos.

8. Repositorio

`https://github.com/Ninaswiftie09/Paralela_Tareas/commits/Laboratorio_SpeedUp/`

A. Gráficas

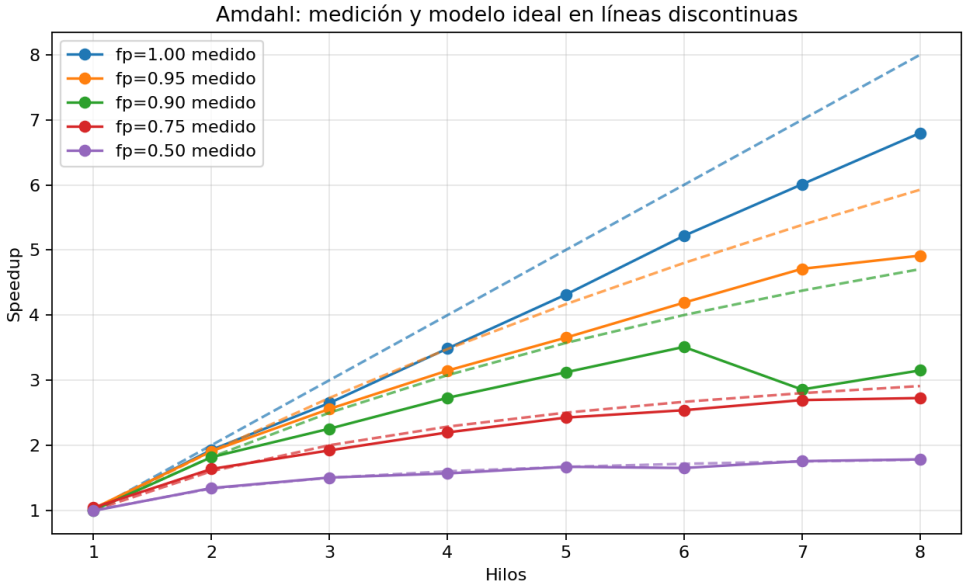


Figura 1: Problema fijo y comparación con el modelo ideal.

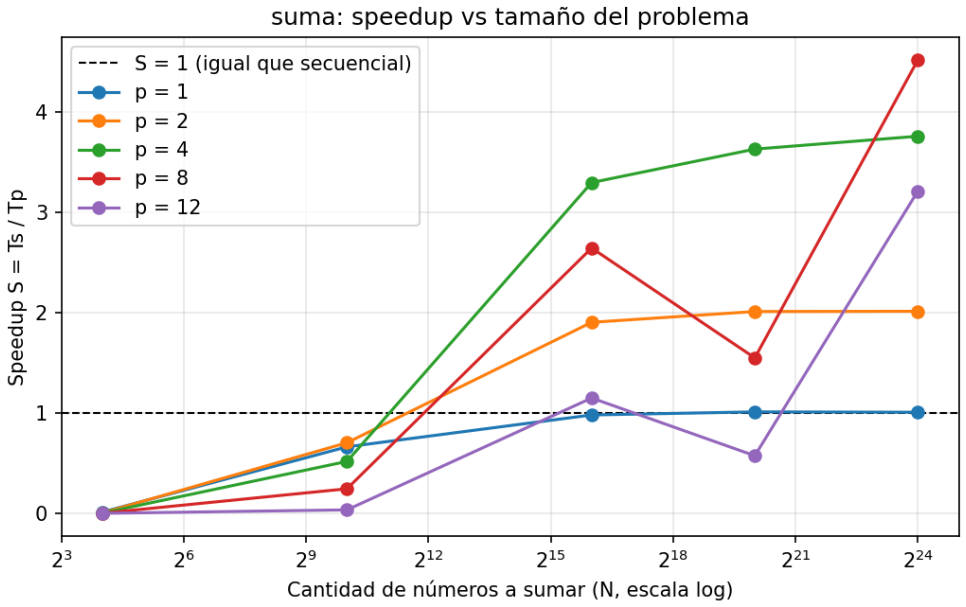


Figura 2: Speedup de la suma según el tamaño del arreglo.

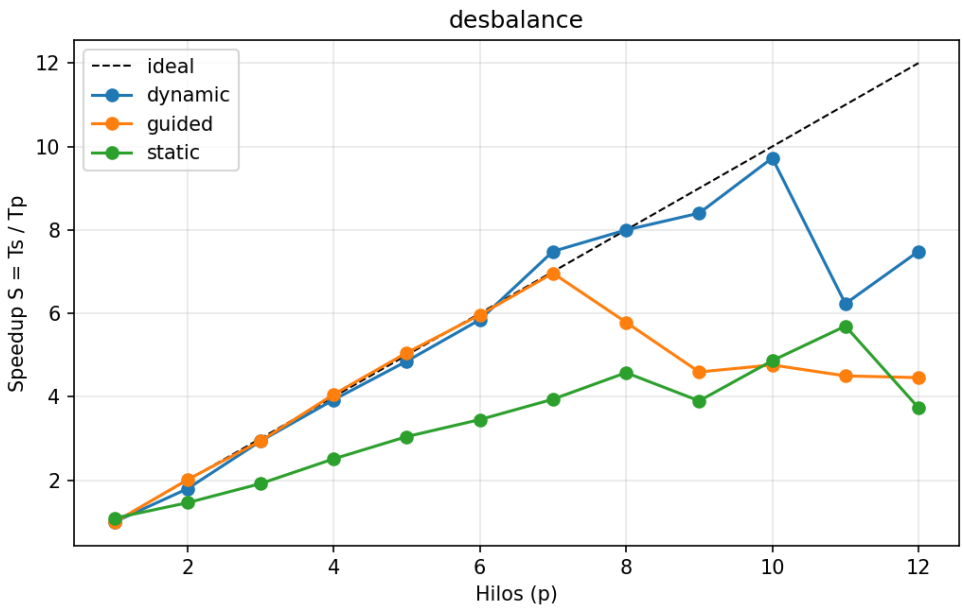


Figura 3: Comparación de los tres repartos de carga.

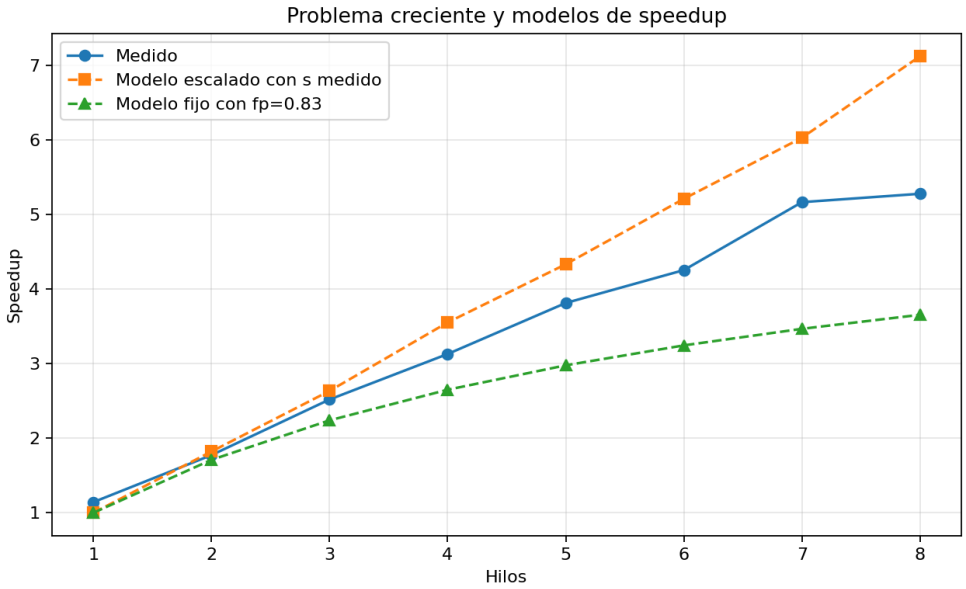


Figura 4: Problema creciente y comparación de modelos.

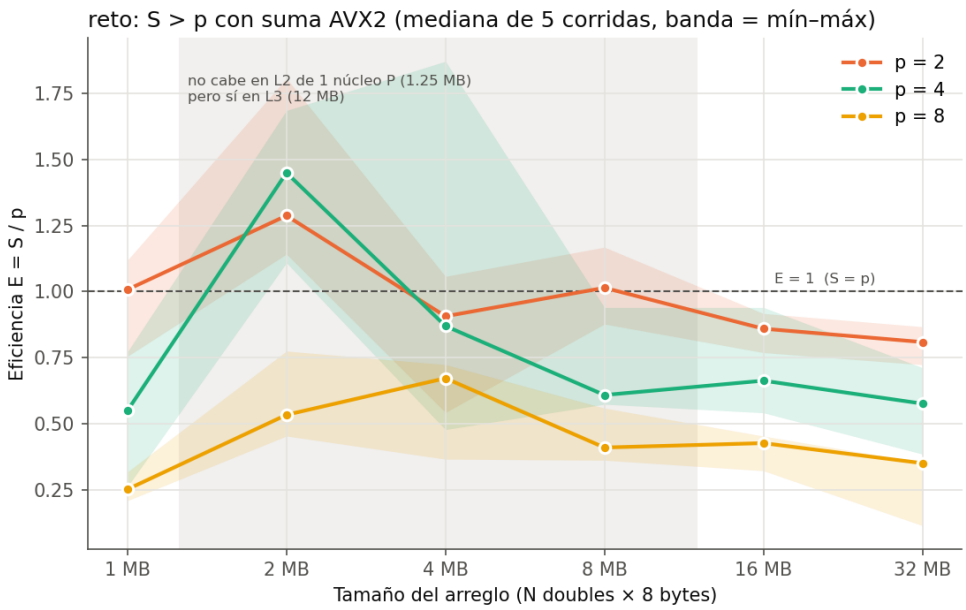


Figura 5: Cinco corridas del reto con suma AVX2.